Knowledge Representation & Symbolic Reasoning Course Project: RoboCup@Home-inspired Tasks

Carlota Alvear Llorente
Group 5
TU Delft
calvearllorent@tudelft.nl

Núria Seguí Vidal
Group 5
TU Delft
nseguividal@tudelft.nl

Panagiotis Sarikas
Group 5
TU Delft
psarikas@tudelft.nl

***Abstract*—This paper presents a hybrid architecture for autonomous mobile manipulators operating in partially observable, dynamic environments. The approach integrates a semantic knowledge base (TypeDB) with symbolic planning (PDDL and PlanSys2), enabling adaptive task execution beyond static planning. Instead of a global plan, the system generates localized planning problems from an evolving world model built through active perception. The method is evaluated on three RoboCup-inspired tasks: spatial tidying, semantic tidying, and contextual object retrieval with obstacle handling. Results show improved robustness, scalability, and planning efficiency through the combination of semantic inference and modular multi-PDDL planning.**

I. Introduction

This project addresses a Knowledge Representation and Reasoning (KRR) problem in a partially observable and dynamic environment. A mobile manipulator robot is assigned with tasks that involve autonomous exploration, semantic classification and object manipulation across multiple rooms, while respecting semantic relationships such as object-to-drop mappings and contextual inference rules.

A key challenge is the absence of prior knowledge: the robot must continuously adapt to newly discovered objects and obstacles (e.g., blocked doorways), making static planning approaches unsuitable. To overcome this, a hybrid cognitive architecture that integrates semantic reasoning with symbolic planning is proposed.

A centralized knowledge base, implemented in TypeDB, maintains a dynamic representation of the environment and supports rule-based inference (e.g., predicting object locations from context). Task planning and execution are handled using PDDL with PlanSys2 [1]. Instead of generating a global plan, the system dynamically constructs localized planning problems based on the robot's current knowledge, enabling real-time, context-aware decision making.

The proposed architecture (see Figure 1) operates in three continuous phases:

- **Knowledge Acquisition:** The robot explores the environment and updates the knowledge base with observed objects, obstacles, and drop locations.
- **Planning:** A Task Manager queries the knowledge base and formulates a PDDL problem based on the current state and objective.
- **Execution:** PlanSys2 executes the plan by mapping symbolic actions to low-level robotic skills such as navigation, grasping, and placing.

This modular design improves scalability and adaptability, as environment-specific changes are handled directly in TypeDB.

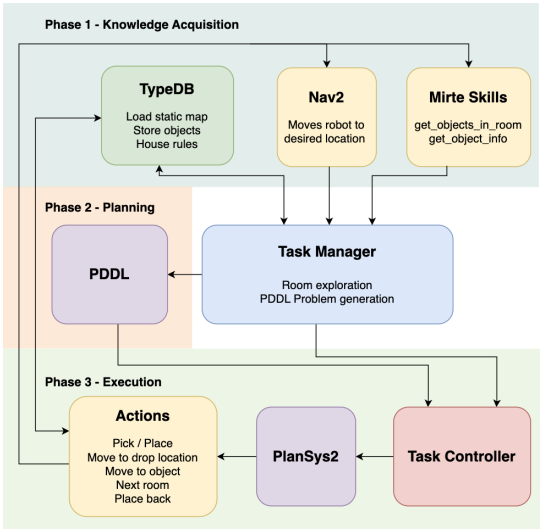

Fig. 1. General system architecture mapping Phase 1 (Acquisition), Phase 2 (Planning), and Phase 3 (Execution).

To evaluate the approach, the architecture is tested in three progressively complex tasks:

- **Task 1 (Simple Tidy Up):** Place each object in the closest drop location within the same room.
- **Task 2 (Semantic Tidy Up):** Place objects according to semantic rules (e.g., *dirty tableware* $\rightarrow$ *dishwasher*).
- **Task 3 (Find and Bring):** Locate a target object using contextual inference and clear obstacles in the way. Then place remaining objects according to semantic rules (task 2).

II. Methodology

As mentioned in Section I, the project is modeled as a hybrid reasoning architecture in which *TypeDB* maintains a semantic and geometric world model, while *PDDL* and *PlanSys2* solve the action-sequencing problem. The common or-

chestration layer is the `TaskManagerBase`, which provides reusable functions for navigation, perception, TypeDB insertion/querying, PDDL problem loading, and plan triggering. Task-specific managers specialize this base class by deciding: (i) the task objective, (ii) which knowledge is available a priori, (iii) what must be perceived online, and (iv) whether the task is solved as one global apartment-level PDDL problem or as a sequence of local room-level problems [2].

Central to this orchestration is a TypeDB schema that organizes knowledge into static entities (`room`, `location`), loaded at initiation, and dynamic `item` types, like detected objects, their poses, and attributes, which are refined at runtime. Relations like `physical-location` map these entities to poses and rooms, while inference rules (`correct-drop`, `book-clue`) enable the system to derive mission goals from the observed context. For example, during the `pick` action, the object identity is discovered and updated using `/get_object_info`, enabling semantic reasoning for Tasks 2 and 3.

Across all tasks, the PDDL state uses a compact symbolic vocabulary with three core types: `item`, `location`, and `room`, denoted as `i`, `l` and `r` respectively. The main predicates are: `item_at(i,l)`, `location_in_room(l,r)`, `scan_loc(l)`, `adjacent(r1,r2)`, `agent_at(l)`, `free_gripper`, `busy_gripper`, `in_gripper(i)`, and room-progress predicates `tidying(r)`, `untidy(r)`, and `tidy(r)`. This abstraction keeps PDDL focused on symbolic reachability and temporal ordering, while TypeDB provides geometric grounding and semantic inference.

A. Task 1

Task 1 is formulated as a room-local spatial assignment problem followed by symbolic execution. During knowledge acquisition, the robot scans a room and stores the perceived objects and drop locations in TypeDB. The task manager then computes the nearest drop location for each object using Euclidean distance within the same room.

The generated PDDL goals are therefore explicit, taking the form `on_drop_loc(item,drop)`. The planner only needs to determine a valid symbolic sequence of navigation, pick, place, and room-transition actions to satisfy these assignments. Since the placements depend only on local room geometry and its objectives are local to each room, Task 1 is solved efficiently as a sequence of room-level multi-PDDL problems rather than as a single computationally heavy apartment-level problem. In this formulation, drop locations are initially unknown and are discovered online during room scanning.

B. Task 2

Task 2 maintains the same overall three-phase planning structure as Task 1, but the drop location is determined semantically rather than geometrically. Here the destination depends on the object class and attributes according to the house rules. Therefore, the PDDL goal is abstracted to `on_drop_loc(item)` without explicitly naming the destination. The symbolic planner is only responsible for deciding when an object should be picked, transported, and placed.

The actual destination is inferred from TypeDB following object identification. After `pick`, the object is retyped in the knowledge base using the output of the `/get_object_info` service, and the `move_to_drop_location_t2` action node then queries the inferred `correct-drop` relation to retrieve the corresponding destination and pose.This division of responsibilities is the main design choice of Task 2: PDDL performs sequencing, while TypeDB performs semantic destination selection.

Additionally, two Task Manager strategies were implemented: a global **Single PDDL** "scan-then-plan" approach and a modular **Multiple PDDL** approach. Both operate room-by-room, allowing the user to select a specific room to tidy, even if their corresponding drop location lies elsewhere, avoiding strategies that prioritize objects based on drop location proximity.

The **Single PDDL** approach constructs a complete world model before planning and is therefore more suitable when drop locations are not known *a priori*. In contrast, the **Multiple PDDL** strategy assumes that drop locations can be inferred via TypeDB rules. As this inference already requires knowledge of valid drop location entities, it was considered reasonable to also assume access to their positions. Under this assumption, the multiple PDDL approach achieves significantly faster execution times (see Section IV-B), while the single PDDL variant ensures robustness when this assumption does not hold.

C. Task 3

Task 3 extends the multi-PDDL formulation of Task 2 by introducing **contextual search** and **doorway reasoning**. The objective is to locate and deliver `obj_4_book`. As shown in Fig. 2, the Task Manager queries TypeDB for the most likely room via the `book-clue` relation, defaulting to the office if no clues exist. Upon arrival, an *Exploration PDDL* problem is generated to inspect visible objects. If the selected room is non-adjacent and chosen only due to missing clues, the system instead explores adjacent intermediate rooms, enabling incremental information gathering before committing to a distant target.

The execution logic then branches based on the inspection results. If the book is not found, the manager chooses the next room and generates a *Next Room PDDL*, which evaluates blocked transitions (`door_blocked_by(i,r1,r2)`) and clears the doorway if necessary. Once the book is found, it is delivered using a *Task 2 PDDL*. Finally, after placing the book—or if all rooms are explored without success—the system clears any remaining required doorways and tidies the rest of the objects using the Task 2 semantic strategy. If execution fails at any point, the manager performs a recovery scan and updates object poses in TypeDB before replanning.

D. Symbolic action design

The PDDL actions encode *task-level state transitions*, while action nodes execute the corresponding physical skills. The

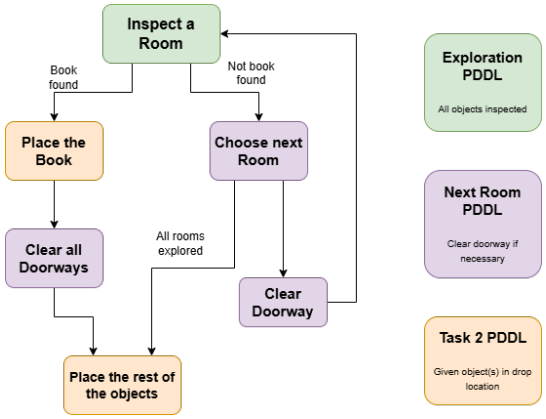

Fig. 2. Task 3 State Machine

navigation action `move_to_object(?i ?l1 ?l2 ?r)` requires the robot at `?l1`, the object at `?l2`, a free gripper, and an active room `tidying(?r)`. Its effect is positional, updating `agent_at` to represent symbolic approach.

The action `pick(?i ?l ?r)` models grasping: it requires co-location, a free gripper, and room consistency, and results in `in_gripper(?i)` while updating gripper state. In Task 3, it also marks `inspected(?i)` and `just_picked(?i)`, treating grasping as inspection. A variant, `pick_obstacle`, also clears blocked doors by replacing `door_blocked_by` with `door_clear`.

In Task 1, `move_to_drop_location(?i ?l1 ?l2 ?r)` explicitly navigates from the current location to a known drop location `?l2`; its preconditions require `drop_loc(?l2)` and `in_gripper(?i)`, and its effect updates `agent_at`. In contrast, in Tasks 2 and 3 the drop location is not parameterized in PDDL. The action `move_to_drop_location(?i ?l ?r)` only requires that the robot is holding `?i`; its symbolic effect is to add `ready_to_place(?i)`. The actual destination is resolved in TypeDB inside the corresponding action node. Thus, Task 1 uses fixed symbolic goals, whereas Tasks 2–3 rely on online semantic inference.

The `place(?i ?l ?r)` action requires the robot at the drop location, the item in the gripper, and the location to be a valid drop. Its effects release the object, restore `free_gripper`, and add `on_drop_loc(?i,?l) / on_drop_loc(?i)`.

Finally, `next_room(?r1 ?r2 ?l1 ?l2)` encodes room transitions. It requires an active room and free gripper, and updates `tidying(?r1)` to `tidy(?r1)`, activates `tidying(?r2)`, and moves the robot to the next waypoint. In Task 3, it additionally depends on `door_clear(?r1,?r2)`, linking navigation to doorway reasoning.

III. Implementation

The software architecture uses a two-layer design that separates the mission strategy from the execution. A python-based **Task Manager** node handles the environmental discovery, TypeDB knowledge base updates, and dynamic PDDL problem assembly. Concurrently, a C++ **Task Controller** node interfaces with PlanSys2 to manage plan execution. This separation allows the manager to focus on high-level logic while the controller ensures robust execution in the simulation (see Figure 1 for the complete schema).

POPF was selected for its real-time reliability and speed. While OPTIC supports optimality [3], its search process resulted in planning times exceeding 400 seconds for simple sub-problems. Additionally, utilizing the `-N` flag to return the first feasible plan yielded results comparable to or less efficient than POPF (see Table I). Therefore, POPF was chosen to ensure stable execution, and to try to reduce the planning time, we decreased the size of the search space by restricting the movement action to two main actions: `move_to_object` and `move_to_drop_location`.

A. Task 1

Task 1 serves as the foundation of the system, focusing on autonomous discovery and spatial mapping. The implementation is split into two main components: an object discovery phase managed by `task1_manager.py` and an execution phase coordinated by `task_controller.cpp`.

- *Object Discovery and Exploration:*
  Since the robot starts with minimum prior knowledge of the environment, the task manager first locates all items and drop-off points. This is achieved by calling the `/get_objects_in_room` and `/get_drop_locations` services and storing the results in the knowledge base.
- *Nearest Drop Location:*
  A custom "nearest neighbor" algorithm is used in the task manager to determine the target drop location. Where the manager identifies the closest drop location using euclidean distance for each object in the room and assigns it as a PDDL goal, significantly reducing the planner's complexity.

Nearest neighbor assignment logic in Task 1

```
closest_drop_id = None
min_dist = float('inf')
for drop_id, (dx, dy) in room_drops.items():
    dist = euclidean_distance(pose.position.x, pose.
        position.y, dx, dy)
    if dist < min_dist:
        min_dist = dist
        closest_drop_id = drop_id
object_to_closest_drop[obj_id] = closest_drop_id
```

- *Planning and Action Execution:*
  Once the environment is mapped, the `build_pddl_problem()` function generates the PDDL problem with the discovered items and their pre-calculated destinations. This is handed to the Task Controller, which invokes the C++ action nodes, such as `action_move_to_drop_location_t1.cpp`, to translate TypeDB coordinates into Nav2 goals [4].

B. Task 2

Task 2 transitions from proximity to house rule-based logic, where destinations are determined via TypeDB inference rather than distance.

- *Knowledge Refinement:*
  The node `action_pick.cpp` was modified to update the robot's knowledge upon grasping. After a successful pick, the node calls the `/get_object_info` service and executes a TypeDB `UPDATE` query. This relates a generic `item` into a specific class (e.g., `tableware`) and assigns attributes (e.g., `cleanliness`), enabling the system to apply semantic house rules.

TypeDB update logic in action_pick.cpp

```
"match $i isa item, has id '" + item_id + "'; "
"delete $i isa item; "
"insert $i isa " + typedb_type + ", has id '" + item_id
     + "', has cleanliness '" + cleanliness + "';"
```

- *Goal Definition:*
  As said in Section II-B, the PDDL domain is designed to be independent of the final physical destinations. The planner only needs to know that an item must be placed, but not where, abstracting the PDDL goal to a simple `(on_drop_loc item)`.
- *Getting the Drop Location:*
  To retrieve the physical coordinates of the correct drop location, the `action_move_to_drop_location_t2.cpp` node queries TypeDB for the `correct-drop` relation, before moving. This relation is automatically inferred by rules defined in the schema (e.g., *dirty tableware belongs in the dishwasher*) the moment the object's class is updated.

Semantic destination query in Task 2

```
match $i isa item, has id 'obj_0';
$d isa drop-location;
(dropped-item: $i, target-location: $d) isa correct
     -drop;
get $d;
```

C. Task 3

Task 3 extends the architecture by adding contextual search logic and new actions to help the robot manage blocked paths.

- *Clue Processing Logic:*
  In `task3_manager.py`, the `likely_room()` function prioritizes the robot's exploration path. It queries TypeDB for the `book-clue` relation to identify the most probable location for the target book. If no clue is found, the code implements a fallback logic that defaults the search to the `office`. This informed hypothesis is then used to reorder the room navigation sequence.

Contextual search and fallback in task3_manager.py

```
def likely_room(self, unvisited_rooms: list) -> str:
    # Query TypeDB for clue-based location
    query = 'match $b isa book... (target-book: $b,
         likely-room: $r) isa book-clue; get $r_name;'
    # Fallback to office if no results found
    return 'office' if 'office' in unvisited_rooms else
          unvisited_rooms[0]
```

- *Doorway Obstacle Processing:*
  Obstacles are identified via `_process_doorway_obstacles()`, which updates TypeDB with `door_blocked_by` predicates. A specialized `place_back` action is used to resolve these blockages, by sending a rotation command to the `/cmd_vel` topic, turning the robot base 90° away from the doorway before releasing the object. This clears the path locally and satisfies the PDDL `(door_clear)` requirement needed for using that door.
- *Failure Recovery:*
  To handle uncertainties and physical execution failures, the manager utilizes a control loop that monitors the `ActionResult` from PlanSys2. If an action fails (e.g., a failed grasp due to physical drift), the `_recover_and_update_poses()` function triggers a localized recovery scan. This behavior allows the robot to re-perceive the room, update the shifted object coordinates in TypeDB, and regenerate a valid PDDL plan to resume the mission automatically without crashing.

IV. Results

The proposed architecture was evaluated through a series of simulated trials in the given Gazebo environment.

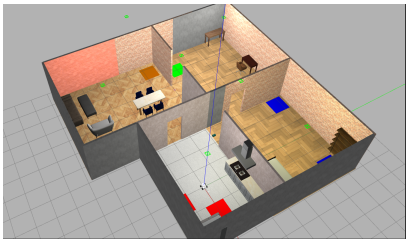

Fig. 3. Gazebo environment showing the four-room layout, target objects (green boxes), and drop locations (colored squares).

While measuring performance based on three metrics: the **Total Completion Time**, defined as the duration from mission initialization to final object placement; the **Total Planning Time**, representing the total time taken by the PDDL planner to generate a valid solution; and the **Success Rate**, the percentage of trials that reached the goal. These metrics provide an overview of the system's efficiency, computational cost and robustness.

A. Task 1

Task 1 served as the baseline for navigation and manipulation. The "nearest neighbor" logic implemented in the python task manager allows us to take away the complexity from the PDDL planner. On average, after 20 trials, the system demonstrated a 95% success rate, as in one of the trials, it failed to pick an object and it led to a simulation crash. As well, to optimize performance, we compared the POPF and OPTIC planners. In general, POPF was preferred because it returns a good feasible solution quickly, whereas OPTIC either takes much longer to find an optimal plan or, with `-N`,

TABLE I
TASK 1 PLANNER COMPARISON (POPF VS. OPTIC)

Approach	Planning Time	Completion Time	Success Rate
POPF	0.98 s	304 s	95 %
OPTIC	1.11 s	269 s	95 %

returns a first feasible plan without clear execution benefits in Tasks 2 and 3. In Task 1, however, this first feasible solution improved performance and is therefore recommended *(Video demonstrations: using OPTIC [5] and using POPF [6]).*

B. Task 2

In task 2 we evaluated the trade-off between the *Single-PDDL* (global) and *Multiple-PDDL* (iterative) implementations. Table II summarizes the average performance across 10 simulation trials.

TABLE II
TASK 2 PERFORMANCE COMPARISON

Approach	Planning Time	Completion Time	Success Rate
Single-PDDL	0.234 s	402.0 s	100%
Multiple-PDDL	0.838 s	380.7 s	100%

The data reveals that the Multiple-PDDL approach outperformed the Single-PDDL in execution optimality, saving an average of approximately **22 seconds** per mission. We attribute this to the reduction in unproductive discovery travel. So, by focusing on one room at a time, the system avoids unnecessary trips across the house and moves more optimally to reach distant objects. *(Video demonstrations: using Single-PDDL [7] and using Multiple-PDDL [8]).*

However, the total planning time was higher for the Multiple-PDDL approach (0.838s vs 0.234s), reflecting the accumulated time spent by the task manager to generate, validate, and solve localized problems for each room. But it can be ignored compared to the 22 seconds of time saved at the end. Also both methods achieved a perfect success rate, confirming the reliability of the architecture and logic of the code.

Beyond raw time metrics, we consider that the Multiple-PDDL approach offers superior scalability. Because in larger environments with an increasing number of rooms or objects, a Single-PDDL formulation could suffer from a combinatorial explosion, which is more computationally expensive. By decomposing the task into small room problems, our architecture allows the robot to handle larger environments with more rooms without slowing down.

C. Task 3

Task 3 evaluated the system's ability to integrate new evidence when exploring the house to find a book. It is important to note that the priority has been to achieve logical robustness and failure recovery over speed.

Firstly, task logs verify that the manager successfully replans the room navigation order when it finds clues. And as a design choice, the robot also scans and searches intermediate rooms along the navigation to non-adjacent rooms, to allow for additional clue gathering. This showed that, even though it increases the total execution time, it ensures logical robustness making the algorithm more optimal.

Additionally, the `place_back` action resolved all doorway blockages. By executing a 90° rotation before dropping the object, the C++ node ensured the obstacle was placed outside of the path using a very simple but effective implementation.

Fig. 4. Place back action

Finally, the "Fidget Spinner" case study validates the failure recovery algorithm. Due to its physics, the fidget spinner often drifted after being placed on the floor (this effect can be clearly seen at the end of the Video [9]). And rather than terminating the simulation after a picking error, the 360° recovery scan, explained in section III-C, allowed the robot to correct the object locations and pick them again, achieving a task success rate of 100%. *(Task 3 video demonstration [9]).*

V. DISCUSSION

The results demonstrate that a hybrid KRR architecture successfully manages the trade-off between semantic depth and execution efficiency. However, several limitations were identified during development.

A primary methodological constraint is the robot's lack of high-level spatial awareness, as it relies on hardcoded room waypoints rather than room polygons to define environment boundaries. The software also presents two bottlenecks: the lack of support for numeric fluents, which prevented PDDL from optimizing paths using real-time Euclidean distances [10], and TypeDB connection delays, which occasionally desynchronized the world model from the Gazebo simulation during high-speed reasoning loops.

Another problem was observed in task 3, where Gazebo's physics (e.g., objects rolling back into a doorway after being placed away) can re-block a doorway after it is cleared. Because PDDL assumes action effects are permanent, the robot may attempt to cross a re-blocked path. To fix this, a more robust solution would require verifying the doorway state immediately before crossing rather than relying on a static symbolic predicate.

Finally, for future improvements, the work should focus on standardizing the Failure Recovery Machine across all tasks. Because Task 3 demonstrated that a 360° recovery scan after failing an action allows us to achieve a 100% success rate, so bringing this to Tasks 1 and 2 would resolve the observed 5% failure rate. Additionally, integrating real-time distances into PDDL would enable the robot to use more optimal plans, reducing battery use and travel time.

REFERENCES

[1] F. Martín, J. Ginés Clavero, V. Matellán, and F. J. Rodríguez, "PlanSys2: A Planning System Framework for ROS2," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 9742–9749, 2021. doi: 10.1109/IROS51168.2021.9636544.

[2] A. Lindsay, "On Using Action Inheritance and Modularity in PDDL Domain Modelling," in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 33, no. 1, pp. 259–267, 2023. doi: 10.1609/icaps.v33i1.27203.

[3] Y. Carreno, R. Petrick, and Y. Petillot, "Towards Long-Term Autonomy Based on Temporal Planning," in *Towards Autonomous Robotic Systems (TAROS)*, pp. 143–154, 2019. doi: 10.1007/978-3-030-25332-5_13.

[4] S. Macenski, F. Martín, R. White, J. Clavero. The Marathon 2: A Navigation System. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020.

[5] "Task 1 Execution using OPTIC planner," [Online Video]

[6] "Task 1 Execution using POPF planner," [Online Video].

[7] "Task 2 using the Single-PDDL approach," [Online Video].

[8] "Task 2 using the Multiple-PDDL approach," [Online Video].

[9] "Task 3: Book search," [Online Video].

[10] S. Izquierdo-Badiola, G. Canal, G. Alenyà, C. Rizzo, and A. Coles, "Planning for Human-Robot Collaboration Scenarios with Heterogeneous Costs and Durations," in *Frontiers in Artificial Intelligence and Applications, Vol. 392: ECAI 2024*, pp. 4410–4417, 2024. doi: 10.3233/FAIA241019.

[11] "Package repository," git@gitlab.tudelft.nl:cor/ro47014/students-2526/group_05/krr_agent.git.